



DHBW

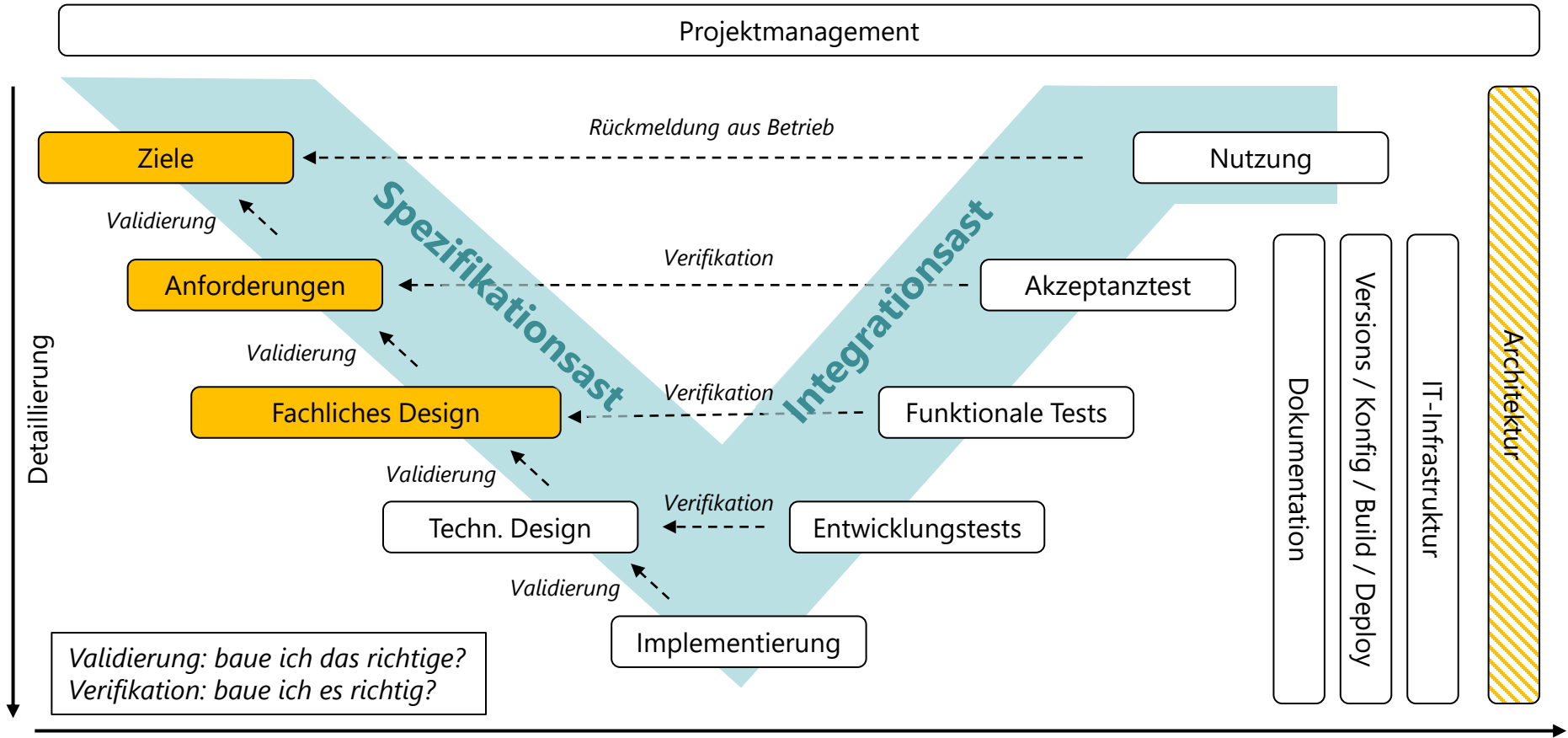
Mannheim

Kapitel 03: Spezifikation

WWI-SE Software Engineering II – Kernkonzepte
Oliver Theißen

www.dhbw-mannheim.de

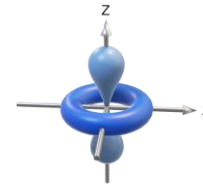
Spezifikation



Bestandteile einer Spezifikation

**Kontext**

- Systeme
- Akteure

**Modellbildung**

- Begriffe der realen Welt
- Regeln der realen Welt

**Ziele**

- Abgrenzung
- Roadmap

**Architektur**

(Nächstes Semester)

**Funktion**

Interaktion mit dem System

- Anwendungsfälle
- User Stories

Ziele



Ziele



Unverzichtbar um Klarheit über die Aufgabe zu bekommen

■ Welche **geschäftlichen** Ziele werden mit dem Projekt verfolgt?

- Gibt es noch Nebenziele?
- Gibt es unausgesprochene Ziele?

■ Was gehört **nicht** zu den Zielen?

- Ohne Abgrenzung “fasert” ein Projekt aus und wird zum Auffangbecken aller sonstigen Wünsche in der Organisation
- Oft klärt sich erst durch die Abgrenzung, was Gegenstand eines Projektes ist und was nicht
 - Beteiligte nehmen stillschweigend an, das mit einem Ziel alle ihre Wünsche abgedeckt sind

■ Roadmap

- **Welche** Ziele sollen **wann** erreicht werden? Das gibt einen ersten Projektplan
- Dabei kommen **Abhängigkeiten** zutage (Abhängigkeiten meistern ist die Wahre Herausforderung beim Software Engineering)

■ IT-spezifisches

- **Nichtfunktionale Anforderungen**. Z.B.: Verfügbarkeit, Anzahl der Nutzer / Skalierbarkeit, Sicherheit, Barrierefreiheit
- **Regulierungen**: Gesetze, Normen, Vorgaben der Organisation
- **Betrieb**: Wer Betreibt die Anwendung? Wer macht den Support? Soll was übergeben werden?



Ziele

Beispiel NGHR

■ Ziele

- Gehaltsabrechnung für Mitarbeiter in allen Ländern erstellen
- Self-Service für Mitarbeiter zum Ändern von eigenen Daten und Einsehen der Gehaltsabrechnung
- Bonuszahlungen an Manager berechnen und in die Gehaltsabrechnung aufnehmen
- Ablösung von Personal Zentral (PZ)

■ Ausschlüsse

- Qualifikationsprofile der Mitarbeiter
- Mitarbeiterentwicklung

■ Roadmap

- Release 1: Mitarbeiterstammdaten
- Release 2: Gehaltsabrechnung für Deutschland für nach Gehalt bezahlte Mitarbeiter
- Release 3: Gehaltsabrechnung für Deutschland für Vertrieb (Provision) und Manager
- Release ≥ 4 : Gehaltsabrechnung in anderen Ländern

■ IT-spezifisches

- Bis zu 1000 Anwender zugleich angemeldet
- Ungeplanter Ausfall nicht mehr als ein Arbeitstag & Gehaltsabrechnung muss ggf. „Nachgeholt“ werden
- GDPR-Konform

Kontext





Kontext

Der (System) Kontext ist die oberste Ebene der Spezifikation

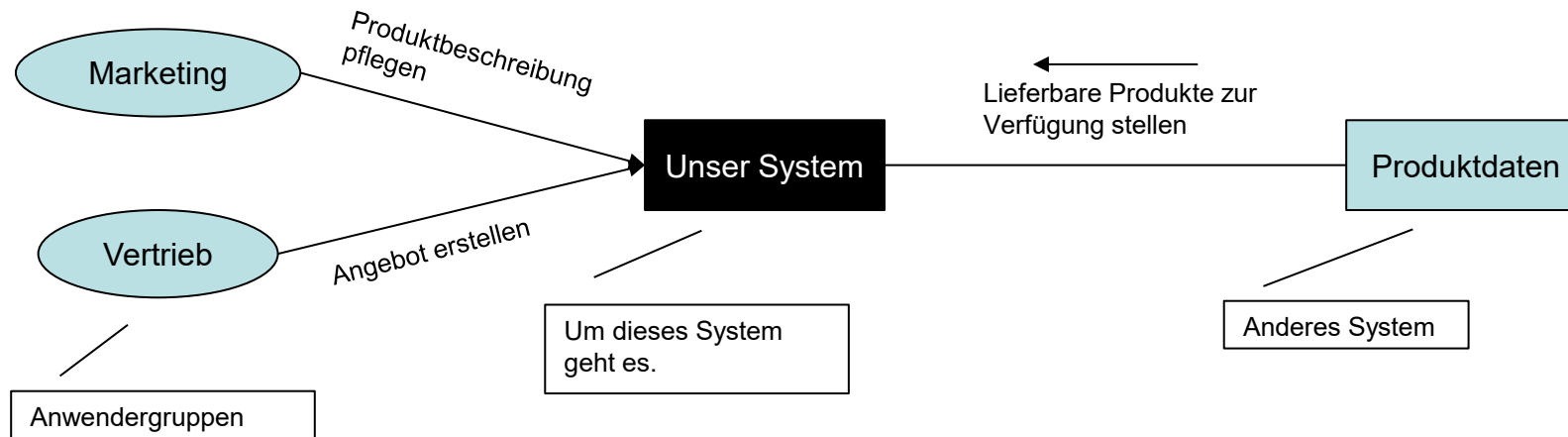
- Betrachte die zu bauende Anwendung als “Black Box”
- Identifiziere **alle Akteure**, die mit der Anwendung interagieren:
 - **Andere Anwendungen**
 - **Anwendergruppen**
 - Akteure können auch passiv mit der Anwendung Interagieren: Nachrichten empfangen (z.B. E-Mail) oder an einer Schnittstelle aufgerufen werden
- Beschreibe die **Interaktion** der Akteure mit der Anwendung:
 - **Tätigkeiten.**
 - **Datenfluss:** Quelle, Ziel, Auslöser, Daten.
 - Hier ist der fachliche Datenfluss gemeint und nicht die Richtung des technischen Aufrufs einer Schnittstelle.
- **Darstellung**
 - Grafisch: als (UML) Kontextdiagramm.
 - Text: Kurzbeschreibungen der Akteure und der Art ihrer Interaktion mit dem System
 - Tabellarisch: für die Datenflüsse.

Kontext



Kontextdiagramm (I)

Bestandteile und grafische Darstellung



Ein Bild sagt mehr als 1000 Worte, aber ein Bild (Diagramm) reicht nicht:

- Definition der Akteure.
- Beschreibung der Interaktion.
- Datenfluss.



Kontext

Kontextdiagramm (II)

Datenflüsse zwischen Systemen werden am besten tabellarisch dargestellt.

Quellsystem	Zielsystem	Auslöser	Daten
Produktdaten	„Unser System“	Produktdaten geändert	Produktnummer und -Name. Listenpreis des Produkts. Lagerbestand des Produkts.
„Unser System“	Buchhaltung	Angebot gebucht	Angebot mit allen Positionen. Gesamtpreis des Angebots.
...	...	...	...

Kontext



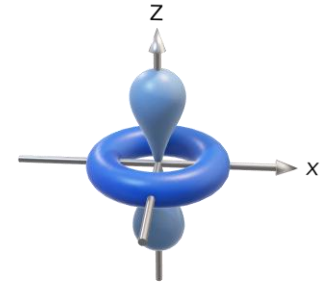
Nutzen und Verwendung

- Definition von **Umfang** und die **Abgrenzung** des Projektes.
- Wir wissen welche **Anwendergruppen** existieren und was diese tun wollen:
 - Mit wem müssen wir im Projekt reden, um die Spezifikation voranzutreiben und für den Anwendertest?
 - Welche Berechtigungsgruppen müssen wir erstellen?
 - Wer muss bei der Inbetriebnahme berücksichtigt werden? Handbuch, Schulung, Information, Berechtigungen.
- Wir wissen welche **Systeme** angebunden werden müssen:
 - Welche Ansprechpartner / Zuarbeit von anderen Teams brauchen wir?
 - Welche Schnittstellen müssen gebaut werden?
- Dokumentation
 - Diskussionsgrundlage während des Projektes.
 - Verwendung für: Einarbeitung neuer Mitarbeiter, IT-Sicherheit, Audits, Betrieb etc.
 - Grundlage für die Darstellung von Prozessen über die Systemlandschaft.

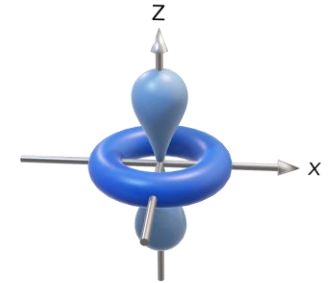
Wenn der Kontext nicht klar ist, dann ist gar nichts klar.

Fehlender oder nicht vereinbarter Kontext ist ein Symptom für mangelnde Klarheit und macht das Scheitern des Projektes wahrscheinlich.

Modellbildung



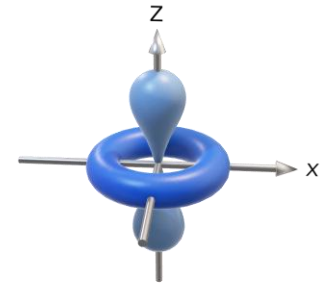
Modellbildung



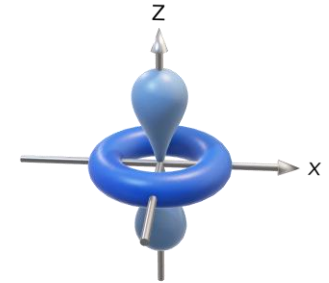
Was ist Modellbildung?

- Es gibt eine reale Welt außerhalb von Systemen und Software: die **Domäne**.
 - Software bildet immer nur eine Untermenge der Domäne ab.
- Diese reale Welt besteht aus:
 - Gegenständen
 - Wissen & Regeln
 - Akteuren und ihre Aktivitäten
- Um Software zu entwickeln, machen wir uns ein **Modell der realen Welt**.
 - Wir machen das im Gehirn. Automatisch. Jede(r) für sich.
 - Es bildet sich im Team eine Sprache heraus, die dieses Modell beschreibt.
- Wenn wir es gut machen, dann schreiben wir dieses Modell auf:
 - **Begriffe definieren** und auch so verwenden – „*Eine Gehaltsabrechnung ist ...*“
 - **Wissen und Regeln** aufschreiben – „*Mitarbeiter sind einer Steuerklasse zugeordnet. Diese Zuordnung erfolgt durch das Finanzamt.*“ oder „*Ein Controller ordnet einem Mitarbeiter eine Kostenstelle zu.*“
 - **Diagramme** für Prozesse, Abläufe, Datenmodell, Zustände...

Modellbildung: Begriffe



Modellbildung: Begriffe



Definition: Begriffe müssen definiert werden

■ Definition von Definition.

- Eine Definition legt einen **Namen** für etwas fest.
- Eine Definition ist **keine Aussage**. Eine Definition kann weder wahr noch falsch sein.
- Eine Definition muss folgende Kriterien erfüllen:
 - **Sinnvoll** sein (man muss mit ihr arbeiten können).
 - Nicht zu **Widersprüchen** führen.

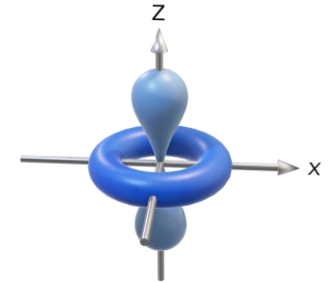
■ Aufbau einer Definition

- **Sprachschablone**: „Ein X ist ein Y, das [Eigenschaften]“
- „Y“ ist ein bereits definierter Begriff.
- Um das „X“ von einem „Y“ abzugrenzen, **qualifizieren** wir das „X“ mit **Eigenschaften**.

■ Regeln für gute Definitionen

- **Minimal**: nur das in die Definition einschließen, was **wesentlich** ist. Sonst schließt man versehentlich etwas aus, was nicht ausgeschlossen sein sollte.
- **Präzise**: eine Definition muss es ermöglichen, zweifelsfrei zu entscheiden ob etwas der Definition genügt (oder nicht).
- **Knapp**: jedes unnötige Wort kann zu Unklarheiten führen. Ein bis drei Sätze.
- Um die Verständlichkeit zu verbessern und Fehlinterpretationen zu vermeiden, kann man zu einer Definition **Erläuterungen hinzufügen**. Diese sollte man von der Definition abgrenzen.

Modellbildung: Begriffe



Beispiel: Definition von Stundenzettel

Stundenzettel dienen zur Erfassung und Abrechnung von Arbeitszeit. Im vorliegenden Fall müssen Stundenzettel von Kostenstellenverantwortlichen genehmigt werden.

Version 1

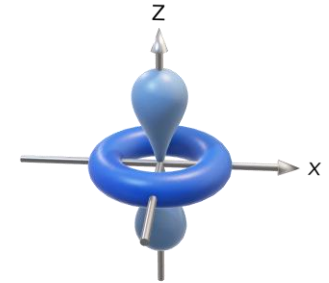
„Die Aufstellung der geleisteten Arbeit wird Stundenzettel genannt. Sie wird von einem Kostenstellenverantwortlichen genehmigt und dann für die Gehaltsabrechnung verwendet.“

Version 2

„Ein Stundenzettel ist eine Aufstellung der geleisteten Arbeitszeit eines Mitarbeiters in einem gegebenen Zeitraum.“

Welche Version ist besser?

Modellbildung: Begriffe

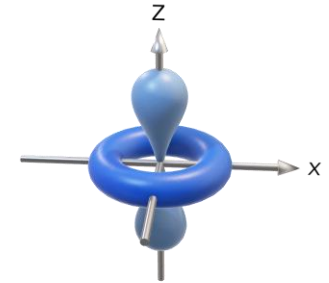


Bewertung von Version 1

„Die Aufstellung der geleisteten Arbeit wird Stundenzettel genannt. Sie wird von einem Kostenstellenverantwortlichen genehmigt und dann für die Gehaltsabrechnung verwendet.“

- Grammatik: das ist keine Definition: „wird genannt“ statt „ist ein“
- Unpräzise: „geleistete Arbeit“
 - Kann die Anzahl der produzierten Werkstücke sein, die Anzahl von Besprechungen etc.
 - Wessen Arbeit ist gemeint? Die der ganzen Firma? Die einer Abteilung?
- Enthält für die Definition unwesentliches: wenn eines der folgenden nicht mehr gültig wäre, wäre es immer noch ein Stundenzettel:
 - Genehmigung durch Kostenstellenverantwortlichen
 - Verwendung in der Gehaltsabrechnung

Modellbildung: Begriffe



Lösung

- Für die Definition Version 2 verwenden.
- Weitere Eigenschaften als Regeln unabhängig von der Definition formulieren.

Definition Stundenzettel

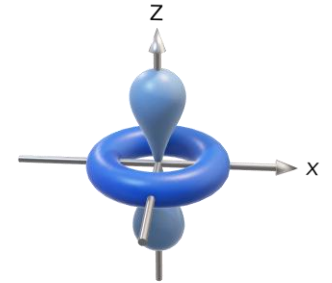
„Ein Stundenzettel ist eine Aufstellung der geleisteten Arbeitszeit eines Mitarbeiters in einem gegebenen Zeitraum.“

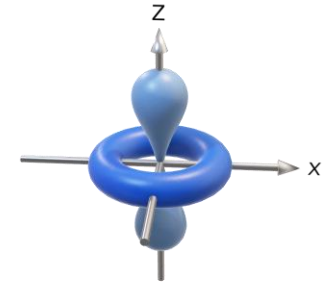
Regeln für Stundenzettel (nicht Teil der Definition)

- Stundenweise bezahlte Mitarbeiter erfassen ihre Arbeitszeit auf Stundenzetteln.
- Der Kostenstellenverantwortliche [...] muss jeden Stundenzettel genehmigen.
- Die Gehaltsabrechnung stundenweise bezahlter Mitarbeiter erfolgt auf Basis der in genehmigten Stundenzettel erfassten Arbeitszeit.

Die Regeln können sich unabhängig von der Definition ändern. Wenn sich die Regeln ändern, bleibt die Definition stabil.

Modellbildung: Fachliches Datenmodell



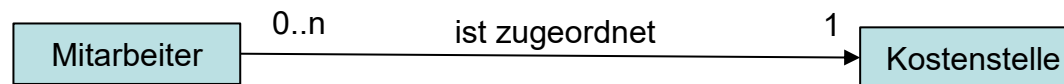


Modellbildung: Fachliches Datenmodell

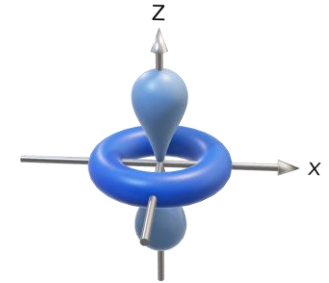
Modell der Geschäftsobjekte und ihrer Beziehungen

Ausführliche Version siehe Vorlesung „Einführung in die Wirtschaftsinformatik“

- Begriffe, die einen Gegenstand der fachlichen Domäne beschreiben sind **Geschäftsobjekte**.
 - Beispiel: Mitarbeiter, Gehaltsabrechnung, Kostenstelle
- Geschäftsobjekte haben **Attribute**.
- Geschäftsobjekte haben **Beziehungen** zueinander.
 - Jeder Mitarbeiter ist genau einer Kostenstelle zugeordnet.
 - Genau ein Manager ist für jede Kostenstelle verantwortlich.
- Eigenschaften der Beziehungen
 - Die Beziehungen haben eine **Bedeutung**
 - Die Beziehungen haben eine **Kardinalität**.
 - Die Beziehungen sind **gerichtet**.



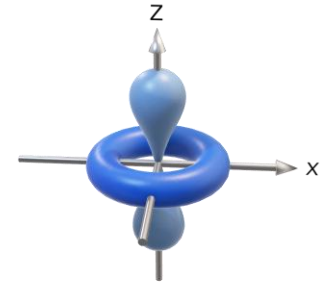
Modellbildung: Fachliches Datenmodell



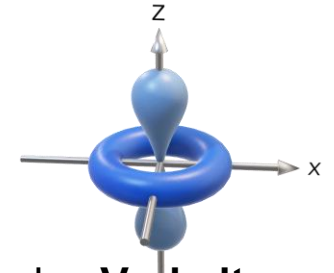
Bemerkungen

- Ein Diagramm sagt mehr als 1000 Worte aber...
 - Definition der Geschäftsobjekte als Text
 - Art der Beziehung beschreiben als Text
- Attribute
 - Beschränkung auf die **Kernattribute** ist oft ausreichend
 - Die wichtigsten Attribute sind **Identifizierer** und **Lebenszyklusstatus**
- Das fachliche Datenmodell ist noch kein vollständiges fachliches Modell
 - Es beschreibt statische Datenstrukturen, nicht das Verhalten
 - Häufiger Fehler: das (einfach zu erstellende) fachliche Datenmodell ist die gesamte Modellbildung
- Logische Datenmodell ist etwas anderes
 - Als Relationen-Modell in einer relationalen Datenbank
 - Als XML/JSON in einer dokumentenorientierten Datenbank

Modellbildung: Regeln



Modellbildung: Regeln

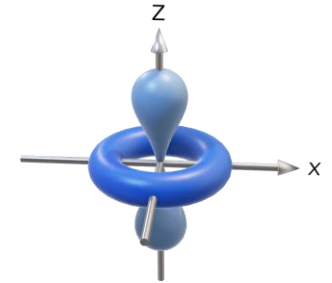


Begriffe definieren die **Gegenstände** eines Modells. Fachliche **Regeln** definieren das **Verhalten** dieser Gegenstände.

- Regeln ohne Begriffe: geht nicht
- Der Mehrwert von Software liegt zum großen Teil im Verhalten
 - Ohne Regeln würde Software nur Daten speichern und abrufen (CRUD – **C**reate **R**ead **U**ppdate **D**eleate)
- Regeln sind oft komplex
 - Können Sie die Regeln, nach denen Ihre Gehaltsabrechnung erstellt wird, aufschreiben?
- Regeln ändern sich häufig. Geschäftsobjekte und Prozesse dagegen seltener
 - Das Geschäftsobjekt Gehaltsabrechnung gibt es schon lange und wird es noch lange geben
 - Den Prozess zur Erstellung der Gehaltsabrechnung ebenso
 - Die Regeln zur Berechnung der Gehaltspositionen ändern sich dauernd

**Die Regeln für die Geschäftsobjekte sind der fachliche Kern von Software.
Sie zu durchdringen, zu modellieren und zu implementieren ist das Herz von
Softwareentwicklung.**

Modellbildung: Regeln



Definition

Eine fachliche Regel ist ein **atomares** Stück **wiederverwendbarer** Entscheidungslogik, das **deklarativ** in **fachlicher Sprache** spezifiziert ist.

Eigenschaft	Erklärung	Nutzen
Atomar	Kann nicht weiter reduziert werden, ohne Bedeutung zu verlieren.	Änderungen auf geringer Granularität.
Wiederverwendbar	Kann für an vielen Stellen für fachliche Aktivitäten und in Systemen verwendet werden.	Hohe Konsistenz.
Deklarativ	Beschreibt <u>nicht</u> das Wie, Wo und Wann ihrer Umsetzung (ist nicht prozedural)	Die Regel bleibt, wenn die Lösung geändert wird.
Fachliche Sprache	In der Sprache der Anwender formuliert. Keine Programmierlogik. Kein Code.	Bessere Kommunikation.

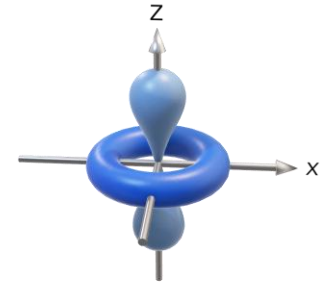
Modellbildung: Regeln

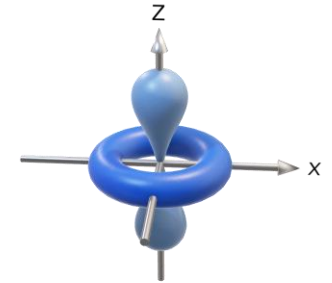
Beispiel: NGHR, Ingmar Intim, Datenschutzbeauftragter

„Grob geht es darum, dass die Gehaltsdaten, Gehaltsabrechnungen und andere personenbezogene Daten von möglichst wenigen Personen eingesehen werden können. Mit Gehaltsdaten meine ich alle Mitarbeiterbezogenen Daten, die zur Erstellung der Gehaltsabrechnung nötig sind.“

So gehen mich die Daten meines Kollegen nichts an. Weder die Privatadresse noch die Bankverbindung und schon gar nicht alles gehaltsrelevante. Die Lohnbuchhaltung darf meine Gehaltsdaten sehen und die Gehaltsabrechnungen. Die Lohnbuchhaltung ist aber wirklich nur die Lohnbuchhaltung und nicht das gesamte Personalwesen. Letzteres hat Einblick in alle anderen Personaldaten, außer Gehaltsdaten und Gehaltsabrechnung.“

Was sind die fachlichen Regeln?





Modellbildung: Regeln

Beispiel: NGHR, Ingmar Intim, Datenschutzbeauftragter

„Grob geht es darum, dass die Gehaltsdaten, Gehaltsabrechnungen und andere personenbezogene Daten von möglichst wenigen Personen eingesehen werden können. Mit Gehaltsdaten meine ich alle Mitarbeiterbezogenen Daten, die zur Erstellung der Gehaltsabrechnung nötig sind.

So gehen mich die Daten meines Kollegen nichts an. Weder die Privatadresse noch die Bankverbindung und schon gar nicht alles gehaltsrelevante. Die Lohnbuchhaltung darf meine Gehaltsdaten sehen und die Gehaltsabrechnungen. Die Lohnbuchhaltung ist aber wirklich nur die Lohnbuchhaltung und nicht das gesamte Personalwesen. Letzteres hat Einblick in alle anderen Personaldaten, außer Gehaltsdaten und Gehaltsabrechnung.“

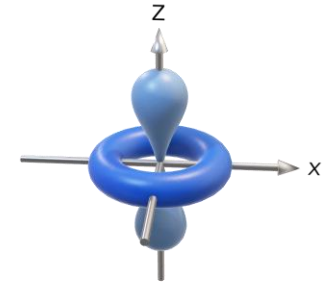
Extrahierte Regeln

- Personenbezogene Daten eines Mitarbeiters sind: Privatadresse, Bankverbindung, Gehaltsdaten
- Mitarbeiter der Lohnbuchhaltung dürfen Gehaltsdaten und Gehaltsabrechnungen aller Mitarbeiter einsehen
- Mitarbeiter des Personalwesens dürfen Privatadresse und Bankverbindung aller Mitarbeiter einsehen
- Mitarbeiter der Lohnbuchhaltung sind Mitarbeiter der Personalwesens

Voraussetzung dass die Regeln klar sind: Begriffe sind geklärt

- „Mitarbeiter“: Sind externe Mitarbeiter eingeschlossen?
- „Gehaltsdaten“ und „Gehaltsabrechnung“: Ist die Gehaltsabrechnung eine Untermenge von Gehaltsdaten?

Modellbildung: Regeln



Grundregeln für Regeln

■ Indikativ. Präsens. Aktiv.

- Die Aktive Formulierung zwingt, das Subjekt zu benennen. Zusätzliche Information. Zusätzliche Klarheit.
- „Der zuständige Recruiter legt die Mitarbeiterstammdaten an.“ statt „Die Mitarbeiterstammdaten werden angelegt.“

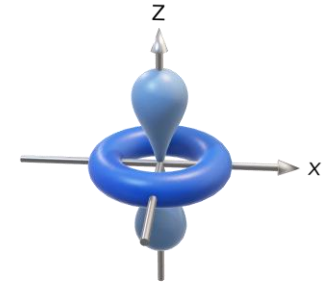
■ Begriffe Referenzieren

- Immer „Stundenweise bezahlter Mitarbeiter“ schreiben, wenn so einer gemeint ist.
- Nicht abkürzen als „Mitarbeiter“, auch wenn es vermeintlich aus dem Kontext klar ist.

■ Dinge und Dinge von Dingen auseinanderhalten

- Der Name eines Mitarbeiters ist nicht der Mitarbeiter.
- Die Personalnummer eines Mitarbeiters ist nicht der Mitarbeiter.

Modellbildung: Regeln



Regeln kombinieren

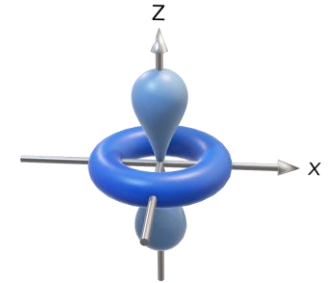
■ Vorsicht mit „oder“ und „und“

- *„Mitarbeiter dürfen nur dann eingestellt werden, wenn eine freie Planstelle existiert und das Budget genehmigt ist oder der Vorstand die Einstellung genehmigt hat.“*
- Was ist die genaue Bedingung?
- Besser: *„Mitarbeiter dürfen nur dann eingestellt werden, wenn mindestens eine der beiden folgenden Bedingungen erfüllt ist:*
 - (1) Es gibt eine freie Planstelle und das Budget ist genehmigt*
 - (2) Der Vorstand hat die Einstellung genehmigt“*

■ „Wenn ... dann“ vermeiden

- Das führt zu prozeduralen (nicht deklarativen) Regeln.
- Schlecht: *„Wenn ein Mitarbeiter eingestellt wurde, dann muss die erste Gehaltsabrechnung angelegt werden und dann muss Controlling darüber informiert werden und dann muss ein User für diesen Mitarbeiter angelegt werden.“*
- Besser: *„Folgendes muss gelten, wenn ein Mitarbeiter eingestellt wurde:*
 - *Die erste Gehaltsabrechnung für diesen Mitarbeiter ist angelegt*
 - *Controlling ist über die Einstellung dieses Mitarbeiters informiert*
 - *Ein User ist für diesen Mitarbeiter angelegt“*

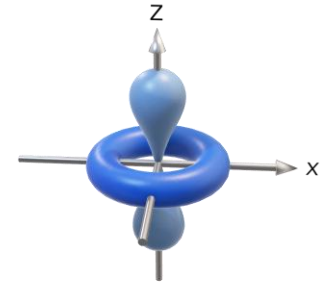
Modellbildung: Regeln

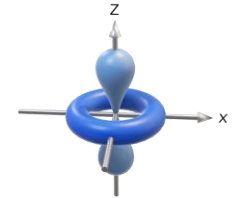


Fehler bei der Formulierung von Regeln

- Etwas Können ist keine Regel. Das ist eine Option.
 - Etwas können heißt nicht zuständig sein oder es ausführen.
 - „Mitarbeiter der Personalabteilung pflegen die Mitarbeiterstammdaten“ statt „Mitarbeiter der Personalabteilung können die Mitarbeiterstammdaten pflegen.“
- Keine impliziten Fakten
 - „Für jedes Team gilt: ein Manager muss das Team managen“ statt „Jedes Team muss einen Manager haben“
 - „Einen Manager haben“ kann heißen, dass dieser Manager irgendetwas mit dem Team macht (coachen, Reisekosten genehmigen,...)
- Fehlende Fakten Vermeiden
 - „Mitarbeiter dürfen nicht während der letzten beiden Wochen eines Jahres eingestellt werden.“
 - Hier ist unklar, ob sich das auf den Zeitpunkt der Unterschrift des Vertrages, auf den Vertragsbeginn oder auf den Verwaltungsakt der Einstellung bezieht.
- Kein „Overkill“
 - Schlecht: „Alle Mitarbeiter des Personalwesens müssen die Stammdaten aller Mitarbeiter immer auf dem neuesten Stand halten.“
 - Besser: „Mitarbeiter des Personalwesens pflegen die Stammdaten der Mitarbeiter.“

Modellbildung: Zustandsautomat





Modellbildung: Zustandsautomat

Ein Zustandsautomat ist eine abstrakte Maschine, die sich zu einem gegebenen Zeitpunkt in genau einem Zustand befindet.

Ist die Anzahl der Zustände endlich, dann spricht man von einem endlichen Zustandsautomaten.

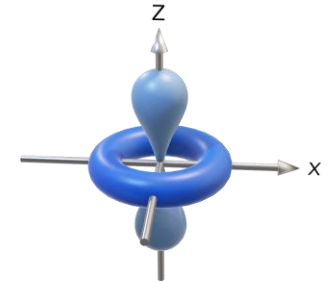
■ Eigenschaften

- Eine (endliche) Anzahl von **Zuständen**.
 - Jeder Zustandsautomat hat genau einen **initialen Zustand**.
- **Ereignisse** können den Zustand ändern (Zustandsübergang).
- **Aktionen** können ausgelöst werden bei:
 - **Verlassen** eines Zustandes
 - **Erreichen** eines Zustandes
 - Einem **Zustandsübergang**

■ Funktionsweise

- Der Zustandsautomat befindet sich immer in einem **Zustand**.
- **Ereignisse** treffen auf den Zustandsautomat in einem Zustand.
- Abhängig von diesem Zustand wird ein **Zustandsübergang** ausgelöst oder nicht.
- Wenn ein Zustandsübergang ausgelöst wurde, kann der Zustandsautomat **Aktionen** auslösen.

Modellbildung: Zustandsautomat



Bemerkungen

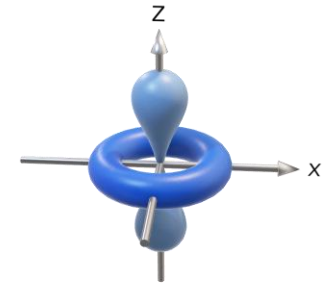
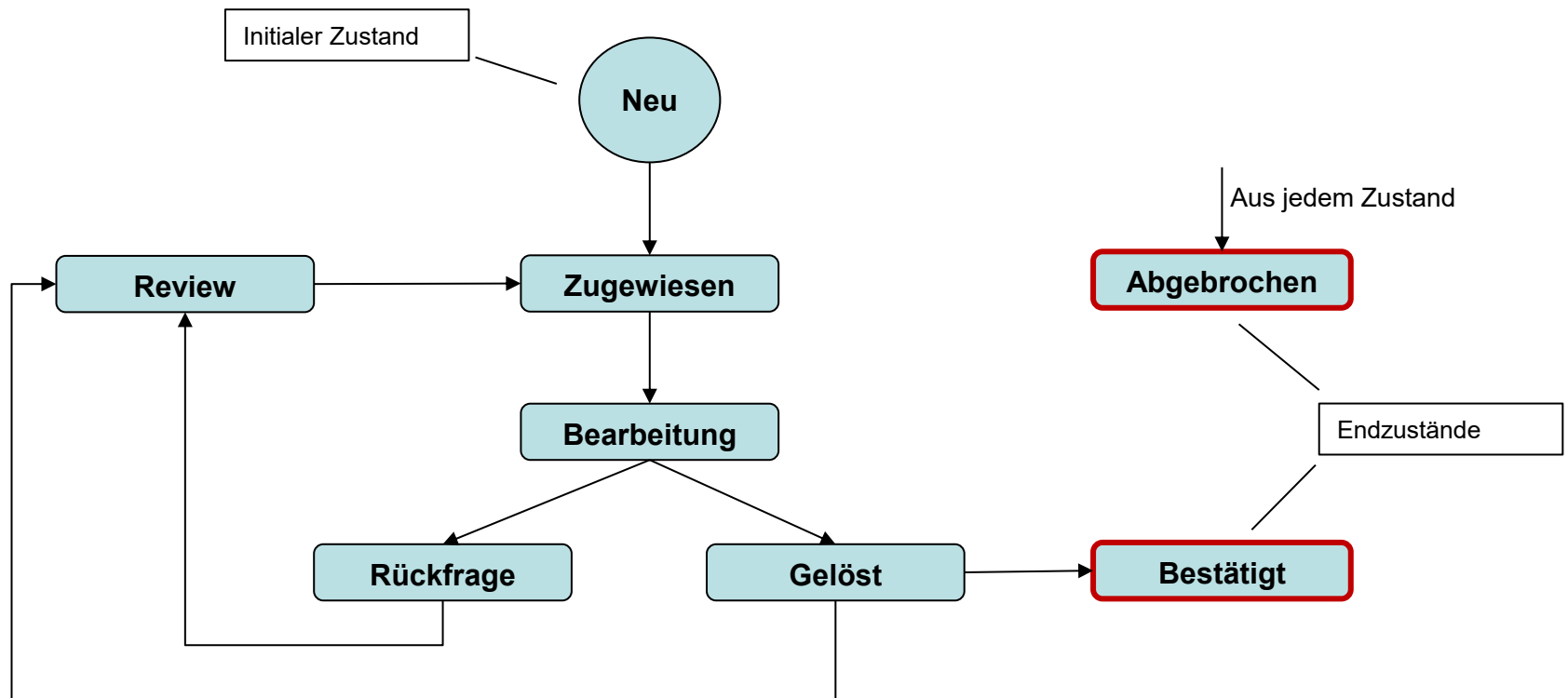
- Eine Methode zum Modellieren von Verhalten
- Allgemein bekannt und damit allen verständlich
- Entkopplung von Zustand, Ereignissen, Zustandsänderungen und Aktionen:
 - Ereignisse können fachlich orientiert erstellt werden (und nicht nach den Zustandsänderungen oder damit verbundenen Aktionen)
 - Der Zustandsautomat weiß selber, was er mit den Ereignissen macht (oder nicht macht)
 - Aktionen sind von den Ereignissen unabhängig → Aktionen sind wiederverwendbar
- Gibt es als Bibliothek für alle relevanten Programmiersprachen

Anwendung in betriebswirtschaftlicher Software

- Modellierung und Implementierung des Lebenszyklus von Geschäftsobjekten.
 - Jedes Geschäftsobjekt hat einen Lebenszyklus: sie sind Teil eines Geschäftsprozesses, der mehrere Schritte hat
 - Der Lebenszyklus neben der Identität die **wichtigste Eigenschaft** eines Geschäftsobjektes
- Entscheidungslogik oder Steuerungslogik

Modellbildung: Zustandsautomat

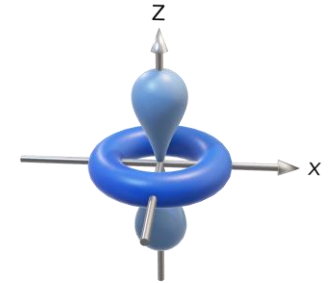
Lebenszyklus eines Support Tickets: Diagramm



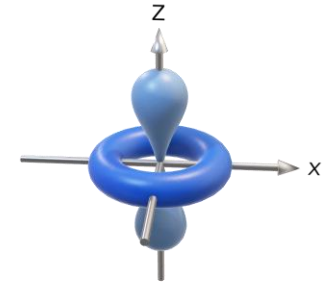
Modellbildung: Zustandsautomat

Lebenszyklus eines Support Tickets: Zustandsübergangstabelle

Als Ergänzung oder Alternative zum Diagramm



Ereignis	Zustand Alt	Zustand Neu
Ticket zuweisen	Neu	Zugewiesen
Ticket zuweisen	Review	Zugewiesen
Bearbeitung beginnen	Zugewiesen	Bearbeitung
Lösung vorgeschlagen	In Bearbeitung	Gelöst
Lösung ablehnen	Gelöst	Review
Lösung bestätigen	Gelöst	Bestätigt
...	...	...



Modellbildung: Zustandsautomat

Lebenszyklus eines Support Tickets: Aktionstabellen

Zustandsübergänge lösen häufig Aktionen im System aus

Variante 1: Aktionen beim **Erreichen** oder **Verlassen** eines Zustandes

Zustand	Aktion Erreichen	Aktion Verlassen
Neu	SLA Timer start	-
Review	SLA Timer start	-
Bearbeitung	-	SLA Timer stopp
...	...	...

Variante 2: Aktionen bei einem **Zustandsübergang**

Zustand Alt	Zustand Neu	Aktion bei Übergang
Neu	Zugewiesen	E-Mail an Bearbeiter: "Neues Ticket"
Review	Zugewiesen	E-Mail an Bearbeiter: "Rückmeldung erhalten"
...	...	...

Modellbildung: Zustandsautomat

Komplexere Varianten

■ Mehrere Zustände pro Geschäftsobjekte

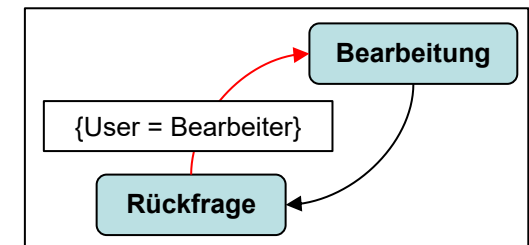
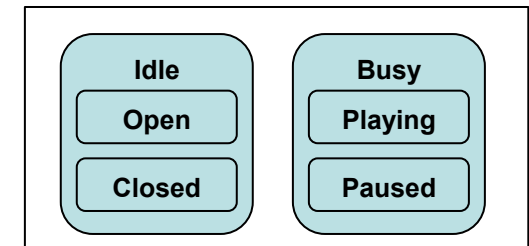
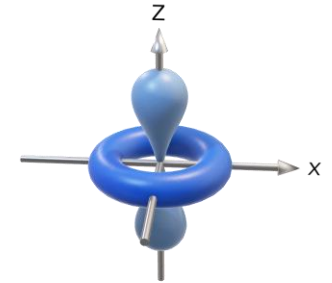
- Bsp.: SIM-Karte
 - **Vertragsstatus** (geschlossen, gültig, gekündigt, beendet)
 - **Technischen Status** (aktiv / inaktiv)

■ Hierarchische Zustandsautomaten

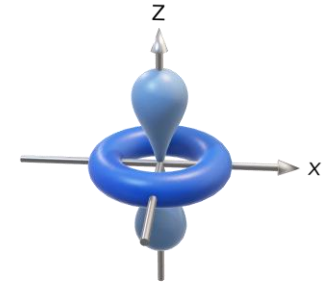
- Bsp.: CD-Player
 - „Idle“ mit Unterzuständen „Open“ und „Closed“
 - „Busy“ mit Unterzuständen „Playing“ und „Paused“

■ Bedingte Zustandsübergänge und Aktionen („Guard Conditions“)

- Bsp.: Support Ticket
 - Wenn Anwender ein Bearbeiter ist, dann ist der Übergang „Rückfrage“ ⇒ „Bearbeitung“ erlaubt
 - Wenn ein Ticket nach der maximalen SLA-Zeit gelöst wurde, dann enthält die E-Mail an den Anwender eine Entschuldigung



Modellbildung: Zustandsautomat



Qualitätskriterium für Zustandsautomaten

Je mehr Geschäftslogik an Zuständen und Zustandsübergängen aufgehängt ist, desto besser und klarer ist die Modellierung

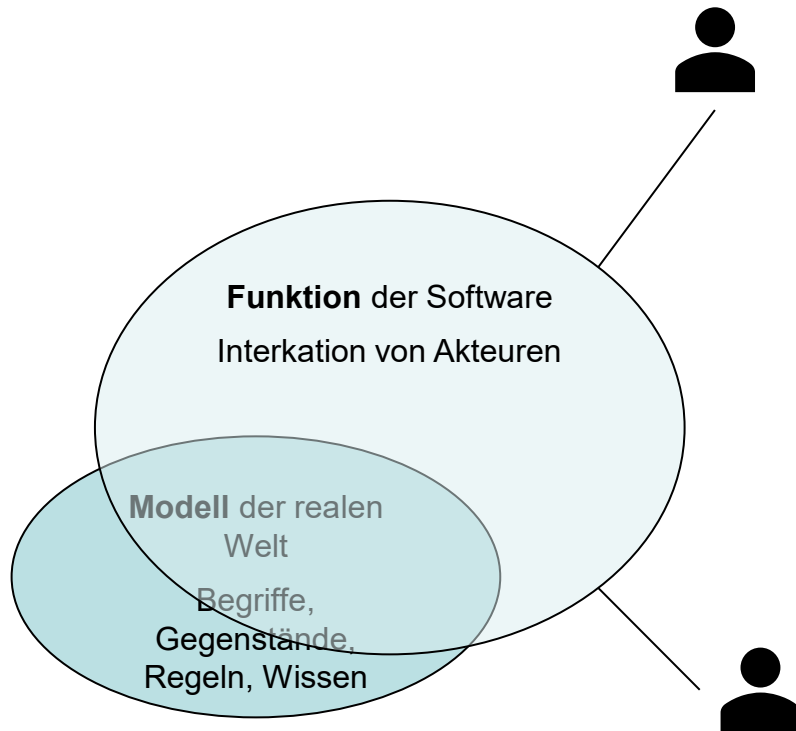
- Man sieht sofort, wo man sich im Geschäftsprozess befindet
 - Entwickler, Fachexperten und Anwender
- Es ist klar
 - welche Bedingungen als erfüllt angenommen werden können
 - welche nächsten Schritte möglich sind
 - wann Nachbedingungen ausgelöst oder erfüllt werden

Funktion





Modell versus Funktion einer Anwendung



Funktion und Modell

- Die **Funktion** beschreibt, wie **Akteure** mit der Software interagieren.
- Das **Modell** beschreibt eine Untermenge der fachlichen Domäne durch Begriffe, Gegenstände, Regeln und Wissen.
- Das Modell ist i.d.R. nicht vollständig in Software abgebildet.
- Das Modell gibt es **unabhängig von Software**. Funktion gibt es **nur mit Software**.
- Zur Spezifikation einer Anwendung wird **beides** benötigt.
 - Ein Modell sagt, nicht, wer und wie es bedient / genutzt wird.
 - Eine Funktion stellt nicht sicher, dass Invarianten^(*) eingehalten werden.

(*)Invarianten: alle relevanten Eigenschaften, deren Wahrheit man prüfen kann.



Anwendungsfall (Use Case)

Definition

Anwendungsfälle beschreiben die Funktionen, die ein System seinen Akteuren bietet

■ Drei Arten von Akteuren

- **Benutzer:** durch Interaktion mit der Benutzerschnittstelle.
- **Externes System:** durch Aufruf einer Schnittstelle.
- **System:** durch internen Auslöser, der nicht durch Interaktion an der Systemgrenze ausgelöst wird. Z.B.: zeitlich gesteuerte Hintergrundverarbeitung.

■ Granularität

- Faustregel: „pro Auslöser ein Anwendungsfall“.
 - Es sei denn wir haben zwei Auslöser und alles andere ist gleich.
- Ein Anwendungsfall kann mehr als einen Akteur haben.
- Wenn Funktionen ausschließlich Nachbedingungen von Anwendungsfällen sind, dann sind sie kein eigener Anwendungsfall.

Alle Anwendungsfälle geben eine vollständige Spezifikation der Funktion einer Software.



Anwendungsfall (Use Case)

■ Verwendung

- Festlegen des Lieferumfangs.
- Implementierungsplanung und Verfolgung.
- Ein Ausgangspunkt zum Identifizieren von Testfällen.

■ Bestandteile

- **Name (Identifizierer)**
- **Akteur:** wer führt den Anwendungsfall durch?
- **Auslöser:** welches fachliche Ereignis löst den Anwendungsfall aus?
- **Vorbedingungen:** welche Bedingung müssen erfüllt sein, das ein Anwendungsfall ausgeführt werden kann?
 - Beispiel: Auslöser ist „Kunde möchte bestellen“. Vorbedingung: Kunde im System angelegt.
- **Ablauf** (prozedural): aus welchen Schritten besteht ein Anwendungsfall und was passiert in jedem der Schritte.
- **Nachbedingungen:** welchen Zustand nimmt das System nach Abschluss des Anwendungsfall ein?
 - Beispiele: „Stundenzettel ist im Status abgerechnet“ oder „Abrechnungsbetrag an Buchhaltung übermittelt“.

■ Weitere mögliche Bestandteile

- Anforderungen
- Anzuwendende Regeln
- Testspezifikation / Abnahmekriterien



Anwendungsfall (Use Case) – Beispiel

ID	UC-EMP-001	
Name	Mitarbeiter Einstellen	Berechtigungsprüfung
Akteur	HR-Mitarbeiter	Könnte auch ein Ereignis im System, einem anderen System oder zeitgesteuert sein.
Auslöser	Arbeitsvertrag mit einer Person geschlossen	
Vorbedingungen	Kein bestehendes Arbeitsverhältnis mit dieser Person	
Ablauf	1. Name und Adresse des Mitarbeiters pflegen 2. Vertragsart pflegen 3. Vertragsstart pflegen 4. Ermitteln der Gehaltsklasse	Validierungslogik
Nachbedingungen	1. Mitarbeiterstatus ist „Vertrag geschlossen“ 2. Erste Gehaltsabrechnung als Hülle angelegt 3. Finanzsystem / Controlling informiert 4. Anlage User ID in Identitätsmanagement.	Ablauflogik
		Auslöser für weitere Aktionen. Ggf. in anderen Anwendungsfällen.



Anwendungsfall (Use Case)

Schwächen beim Arbeiten mit Anwendungsfällen

- Die Anwender denken nicht in Anwendungsfällen:
 - Anwender denken in dem, was sie brauchen.
 - Die Beziehung zwischen Bedarf der Anwender und Anwendungsfällen ist oft n:m.
 - Für die Abstimmung mit Anwendern sind Anwendungsfälle „zu sperrig“.
- Anwendungsfälle neigen dazu, große Dokumente zu werden:
 - Genaue Beschreibung des Ablaufs.
 - Noch dazu: nicht funktionale Anforderung, Abnahmekriterien, Testfälle, Teile des Modells.
- Anwendungsfälle müssen aktuell gehalten werden.
 - Aufwand: erst die Dokumentation aktualisieren und dann den Code anpassen.
- Bei häufigen Änderungen: man wird nie mit einem Use Case fertig.
 - Anwendungsfälle taugen daher nicht für Projektmanagement / Messen des Fortschritts

Wichtig: das sind Probleme mit der Dokumentation eines Anwendungsfalls – der Anwendungsfall (siehe Definition) existiert auch wenn er nicht dokumentiert ist!

- Gute Codequalität: man kann jeden Use Case an einer Stelle im Code finden.



User Story

Definition

„Eine User Story ist eine **informelle Beschreibung** einer Software-Funktion **aus Sicht und in der Sprache des Anwenders**“

- Zweck: eine User Story erklärt,
 - was gebraucht wird und
 - welchen Wert eine Funktion für diesen hat.
- Form: Perspektive und Sprache der Anwender.
 - Kurz gefasst – fast formlos – ohne Details.
- Zusammenarbeit steht im Vordergrund
 - Minimale Dokumentation, um effektive Zusammenarbeit zu ermöglichen
 - Keine vollständige Spezifikation oder Dokumentation.
- User Stories sind kein langlebiges Dokument, das aktualisiert werden müsste.
 - Wird einmal „reif“ für die Implementierung gemacht und dann als erledigt deklariert.
 - Prinzip: Neue Anforderungen = neue User Story.



User Story

Inhalt

Schablone

Ich als [Persona] möchte [Bedarf] sodass [Nutzen]

Akzeptanzkriterien:

[Kriterium 1]

[Kriterium 2]

...

Beispiel

Ich als Mitarbeiter des Recruiting-Teams möchte einen neuen Mitarbeiter einstellen können, sodass sein Name, seine Adresse und seine Gehaltsdaten erfasst sind.

Akzeptanzkriterien:

- Name, Adresse und Gehaltsdaten sind für mich abrufbar
- Die Gehaltsklasse ist festgelegt („Gehalt“, „Gehalt und Provision“ und „Gehalt und Bonus“)
- Die Gehaltszahlungstermine sind festgelegt



User Story

Kriterien für gute User Stories

■ Unabhängig

- Möglichst unabhängig von anderen User Stories: nur dann kann eine User Story unabhängig geplant und umgeplant werden.
- Abhängigkeiten sind unvermeidbar: explizit ausweisen („hat XY zu Voraussetzung“).

■ Verhandelbar

- Lösungsneutrale Formulierung
 - Fördert die Diskussion im Team.
 - Vermeidet, dass der Blick auf eine Lösung verengt wird und bessere Ideen nicht hochkommen.
 - Vereinfachen, aufteilen, umschreiben, zusammenfassen, umplanen soll mögliche bleiben.
- Fokus auf „Was?“ und „Für Wen?“ – nicht auf „Wie?“ und „Wann?“

■ Geschäftswert

- Eine User Story hat immer einen geschäftlichen Wert.
- Grund: diejenigen User Stories, die den größten Wert bringen, werden als nächstes umgesetzt. Ohne geschäftlichen Wert, keine Priorisierung.



User Story

Kriterien für gute User Stories

■ Schätzbar

- Wenn das Team den Aufwand nicht schätzen kann, dann fehlen entscheidende Informationen oder dem Team ist die Umsetzung nicht klar.
- Schätzbarkeit stellt sicher, dass die User Story hinreichend klar ist und dass man mit ihr planen kann.

■ Klein

- Klein genug, um innerhalb einer Iteration / eines Sprints umgesetzt zu werden.
- Zu große User Stories werden aufgeteilt.

■ Testbar

- Überprüfung des geschäftlichen Mehrwertes muss möglich sein.
- Sicherstellen, dass Feedback möglich ist und die User Story eines Tages erledigt ist.
- Qualitätskriterium: wenn man keine Akzeptanzkriterien formulieren kann, dann fehlt etwas.

■ Lieferbar

- Die User Story ist erst fertig, wenn es keine offenen Aufgaben mehr gibt.
- Beispiel: es fehlen automatische Test / Dokumentation / Berechtigungsprüfung / Code Review,...
- Grund: keine versteckten Aufwände & keine technischen Schulden anhäufen.



User Story

Gurken (Gherkins) zur Formulierung von Akzeptanzkriterien

Erfahrungsgemäß tun sich Anwender mit den Akzeptanzkriterien schwer: enthalten oft weitere Anforderungen aber keine Hinweise, wie zu Testen ist.

Abhilfe kann durch Gherkins (Gurken) geschaffen werden: eine einfach Testspezifikation

Schablone

Gegeben [Vorbedingung]

Wenn [Aktivität]

Dann [Ergebnis]

Beispiel

Gegeben ein neu geschlossener Arbeitsvertrag für einen Vertriebsmitarbeiter

wenn der Mitarbeiter eingestellt wird

dann ist die Gehaltsklasse „Gehalt und Provision“

und dann ist der Gehaltszahlungstermin am letzten Wochentag des Monats

- Test ist vorbereitet (braucht man sowieso)
- Fördert die Qualität der User Story: wer Tests spezifiziert schreibt, muss
 - konkret werden. Das fördert die Qualität der User Story und
 - über mehrere (Sonder-) Fälle nachdenken.
- Testspezifikationen sind Beispiele. Beispiele verhindern Missverständnisse.



User Story

Gurkenbeispiel

Beschreibung

Ich **als** HR-Mitarbeiter **möchte** einen neuen Mitarbeiter einstellen können, **sodass** sein Name, seine Adresse und seine Gehaltsdaten erfasst sind.

Die Gehaltsdaten beinhalten die Ermittlung der Gehaltsklasse und des Gehaltszahlungstermins aus dem Vertragstyp.

Testspezifikation

Gegeben ein neu geschlossener Arbeitsvertrag für einen Vertriebsmitarbeiter
wenn der Mitarbeiter eingestellt ist

dann ist die Gehaltsklasse „Gehalt und Provision“

und dann ist der Gehaltszahlungstermin am letzten Wochentag des Monats.

Gegeben ein neu geschlossener Arbeitsvertrag für einen allgemeinen Mitarbeiter
wenn der Mitarbeiter eingestellt ist

dann ist die Gehaltsklasse „Gehalt“

und dann ist der Gehaltszahlungstermin am letzten Wochentag des Monats.

[...]



User Story

Bemerkungen

■ User Stories sind

- **Keine Selbstläufer**: sie müssen in Diskussionen und durch Nachdenken reifen („grooming“)!
- **Keine vollständige Spezifikation**. Es reicht, wenn das Team nach Diskussion(en) zusagt, verstanden zu haben, was zu tun ist.

■ Verbinden Inhalte mit Projektmanagement

- Sie sind die kleinste Einheit, über die man sich bezüglich Priorität, Planung, Entwicklung und Test vereinbart.

■ Können Checklisten enthalten

- „**Definition of Ready**“: allgemeine Kriterien, ob eine User Story zur Planung bereit ist (Abstimmung?, Tester?).
- „**Definition of Done**“: Kriterien um meine User Story als erledigt anzusehen (Getestet?, Release Note?, Code Review?)

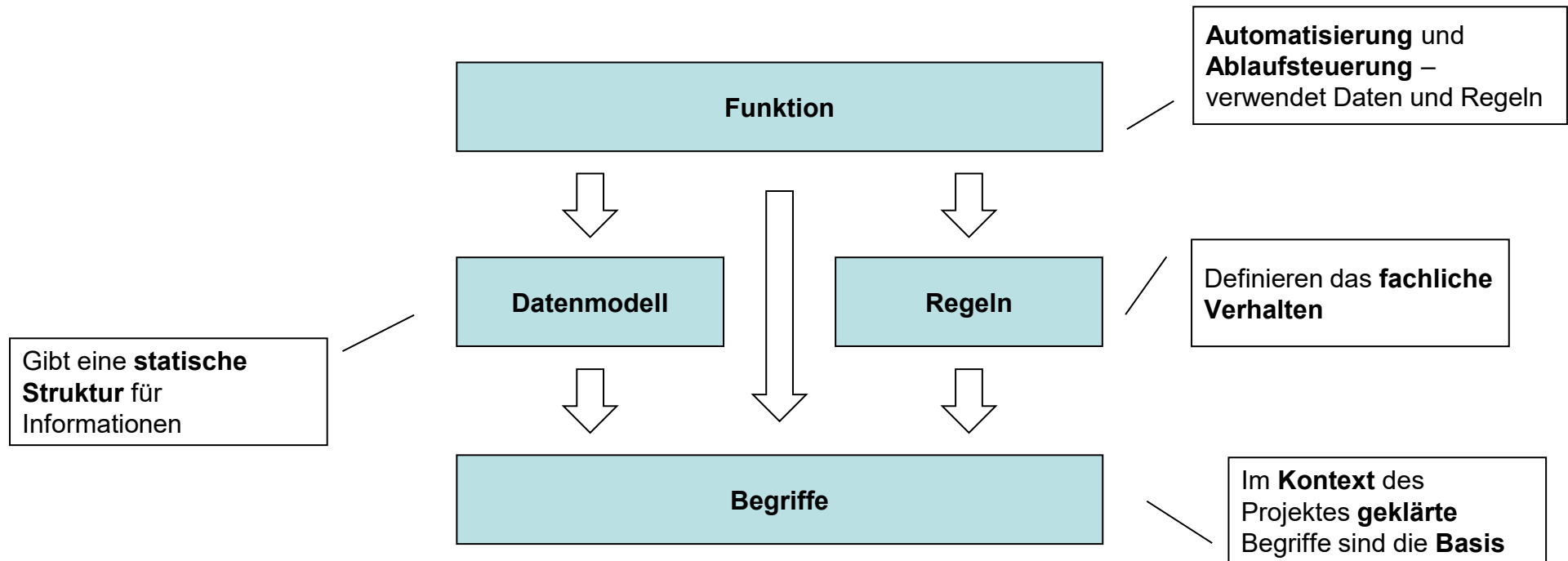
■ Zerlegung und Hierarchien „User Stories“

- **Epic** (Mehrere Releases) – **Feature** (ein Release) – **User Story** (ein Sprint) – **Task** (einige Stunden / Tage)
- Beispiel NGHR:
 - Epic: Employee Self Service
 - Feature: Mitarbeiterstammdaten
 - User Story: Pflege der Privatadresse
 - Task: Berechtigungsprüfung

Zusammenhänge

Zusammenhänge

Begriffe, Regeln, Datenmodell, Funktion

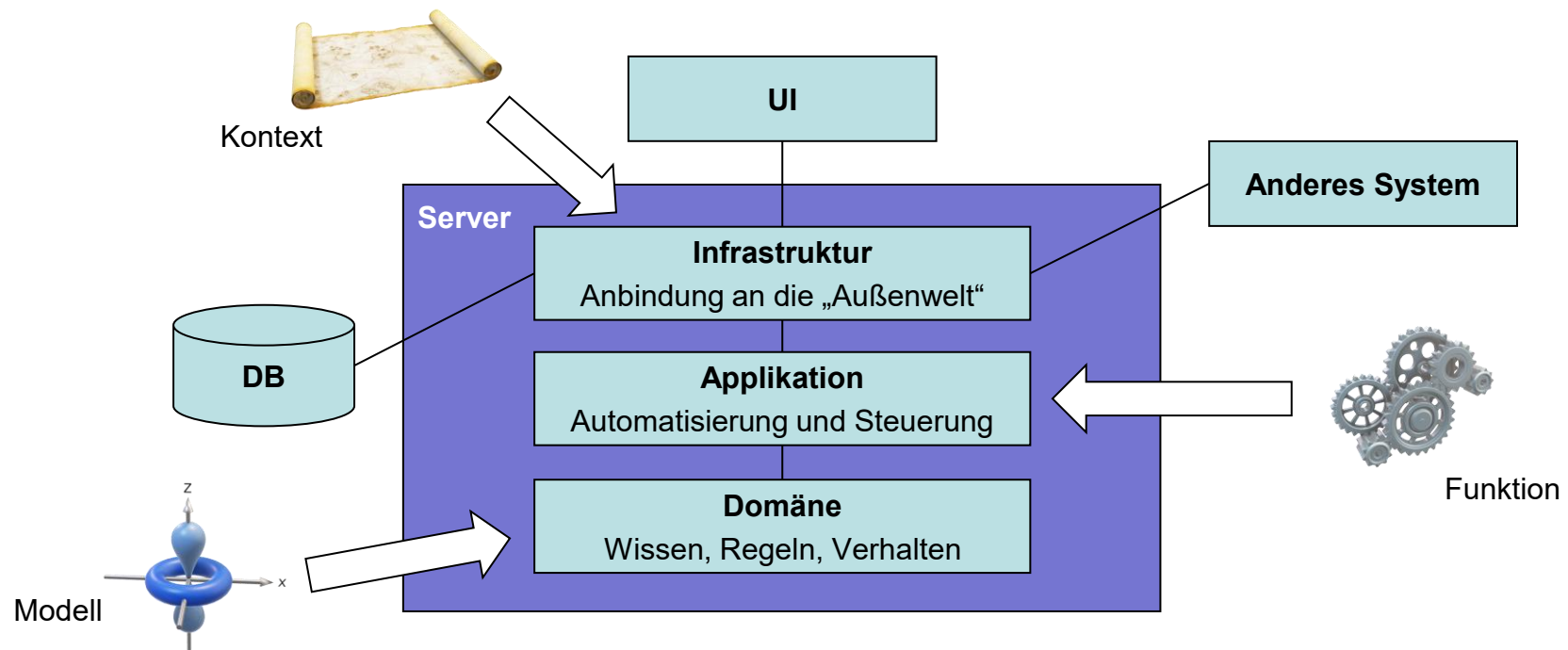


Zusammenhänge



Spezifikation und Architektur: Abbildung der Spezifikation auf den Code

Wir werden den Code innerhalb eines Servers in Infrastruktur, Applikation und Domäne aufteilen



Zusammenhänge

Spezifikation, Kontext und Sprache

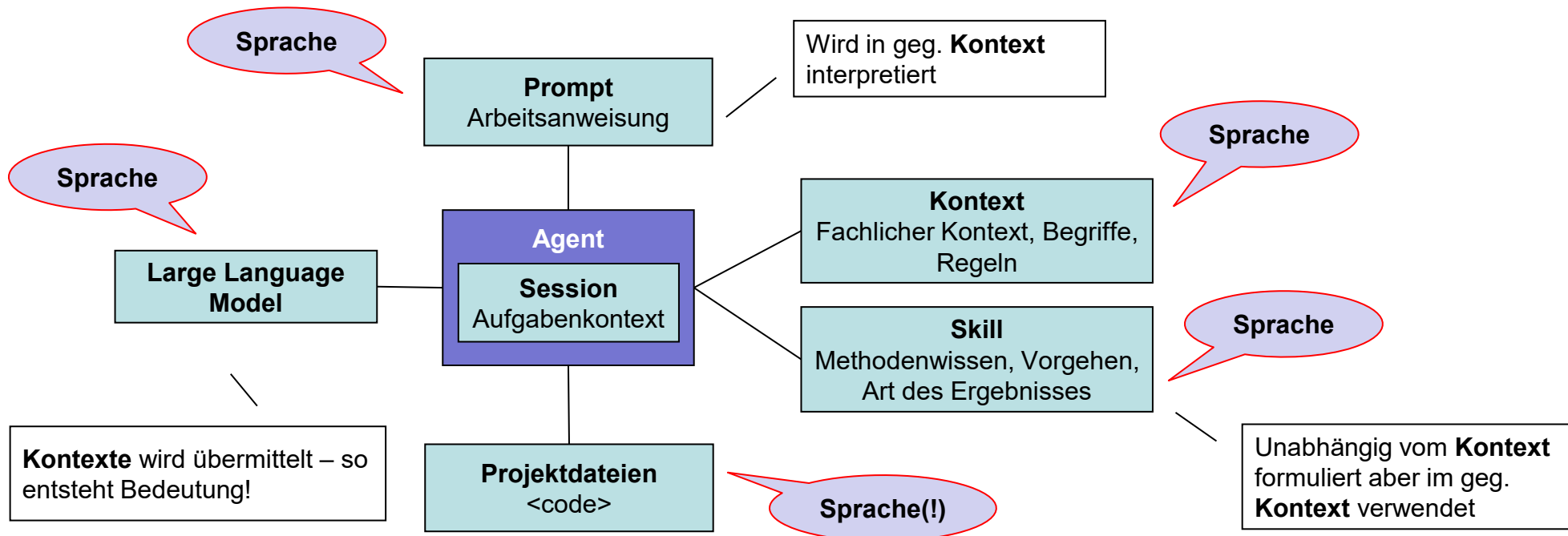
Begriffe, Regeln, Datenmodel, Funktion (Use Case / User Story) sind ganz oder teilweise in **natürlicher Sprache** formuliert

- **Kontext:** Bedeutung gibt es immer in Bezug zu einem Kontext
 - **Alltagskontext:** wir verwenden einen Begriff gemäß unserem üblichen Sprachgebrauch (unscharf, uneinheitlich)
 - „Stundenzettel“ kann Arbeitszeit von Mitarbeitern oder von Beratern erfassen. Was ist nun richtig?
 - **Domänenspezifischer (Sub-) Kontext:** ein geklärter Kontext reduziert Uneindeutigkeiten
 - „Mitarbeiter“ im Kontext Personalwesen bedeutet etwas anderes als „Mitarbeiter“ im Kontext Controlling
- **Fälle:** Plausibilisieren und Ergänzen durch „gesunden Menschenverstand“
 - Wenn uns etwas nicht konsistent erscheint, dann **plausibilisieren** wir
 - Wenn wir eine Lücke wahrnehmen, dann **ergänzen** wir
 - Wir nehmen dabei **vorhandenes Wissen aus dem Alltagskontext** zur Hilfe
 - Damit verlassen wir den domänenspezifischen Kontext!
- **Folgerungen**
 - **Klarstellen**, in welchem **Kontext** ein Begriff / eine Regel / ein Datenobjekt verwendet wird
 - Begriffe **definieren** und Sprache **präzise** im gegebenen **Kontext** verwenden

Zusammenhänge

Kontext, Sprache und KI-Agenten

Referenzarchitektur für einen KI-Agenten für Softwareentwicklung



Zusammenfassung

Zusammenfassung

■ Elemente einer Spezifikation sind:

- **Ziele** – was wollen wir erreichen und was nicht?
- **Kontext** – Oberste Ebene: die Software als “Black Box” mit Input und Output.
- **Modellbildung** – Ohne ein vereinbartes Modell der realen Welt geht es nicht.
- **Funktion** – Festlegen welche Funktionen den Akteuren angeboten werden.
- **Architektur** – Festlegen, wie die Lösung strukturiert wird und was die leisten soll.

■ Kontext

- Akteure (Anwendergruppen und Systeme) mit der Art der Interaktion
- Datenfluss: Objekt – Quelle – Ziel – Auslöser.

■ Modellbildung für den fachlichen Kern

- Definiere **Begriffe** für alle Gegenstände in dem gegebenen **Kontext**.
- Erstelle ein **fachliches Datenmodell** der Geschäftsobjekte (statisch)
- **Die Regeln** legen das Verhalten fest (dynamisch)
- **Zustandsautomaten** als eine Möglichkeit, der formalen Modellierung

■ Funktion für die Beschreibung der Interaktion an der Systemgrenze.

- **Use Case** beschreibt die Interaktion eines Akteurs mit der Software an der Schnittstelle zur Außenwelt.
- **User Story** ist eine informelle Beschreibung einer Software-Funktion aus Sicht und in der Sprache des Anwenders.